

Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents



01 Unary Operators



02 Bitwise Operators



03 Shift Operators



04 Logical Operators



05 Examples

Unary Operators

- Unary operators act on single operands
- Python supports unary minus operator

- Example,

B = 10

A = - B

Print(A)

-10

Bitwise Operators

- These operators perform operations at the bit level
- These include:
 - Bitwise AND
 - Bitwise OR
 - Bitwise XOR
 - Bitwise NOT
 - Bitwise shift operators

Bitwise AND (&)

- It performs AND operation with the corresponding bits
- Ex.

a=2

b=3

c=a & b

print(c) # it will give c=2

Bitwise OR (|)

- It performs OR operation with the corresponding bits
- Ex.

a=2

b=3

c=a | b

print(c) # it will give c=3

Bitwise XOR (^)

- It performs XOR operation with the corresponding bits
- Ex.

a=1

b=2

c=a ^ b

print(c) # it will give c=3

Bitwise NOT (~)

- It performs NOT operation with the corresponding bits
- Ex.

`a = 2`

`b = ~a`

`print(b)` # it will give `b = -3`

Shift Operators

- Python supports two bitwise shift operators
- shift left (<<)
- shift right (>>)
- Ex,

a=2

b=a>>1 # here b=1

Shift Operators

- Python supports two bitwise shift operators
- shift left (<<)
- shift right (>>)
- Ex,

a=2

b=a<<1 # here b=4

Logical Operators

- Python supports logical operators:
 - Logical AND (and)
 - Logical OR (or)
 - Logical NOT (not)

Logical AND (&)

- Logical AND operator is used to simultaneously evaluate two conditions or expressions with relational operators
- If both the conditions are true, then the whole condition is true
- Ex,
(a>b) and (b>c)

Logical AND (&)

- Ex,

a=2

b=3

c=1

(a>b) and (b>c)

False

Logical OR (|)

- Logical OR operator is used to simultaneously evaluate two conditions or expressions with relational operators
- If either of the conditions are true, then the whole condition is true
- Ex,
(a>b) or (b>c)

Logical OR (|)

- Ex,

a = 2

b = 3

c = 1

(a > b) or (b > c)

True

Logical NOT (!)

- Logical NOT operator takes a single expression and negates it

- Ex,

a = 2

b = 3

not (a > b)

True

Membership Operators

- Two types of membership operators:
 - in
 - not in

Membership Operators

- Two types of membership operators
- Ex.

A = [1, 2, 3, 4, 5]

3 *in* A

True

Membership Operators

- Two types of membership operators
- Ex.

msg = "Hello"

'e' **in** msg

True

'H' **not in** msg

False

Identity Operators

- Two types of Identity operators:
 - is
 - is not

Identity Operators

- 'is' operator returns True if operands or values on both sides of the operator

- Ex.

a = 1

b = 1

c = 2

a is b #it shows **True**

b is not c #it shows **True**

Operators Precedence & Associativity

- Relative priorities of the operators w.r.t others is very important
- It determines the order in which the expression is evaluated

Operators Precedence & Associativity

Table 3.7 Operator Precedence Chart

Operator	Description
**	Exponentiation
~, +, -	Complement, unary plus (positive), and minus (negative)
*, /, %, //	Multiply, divide, modulo, and floor division
+, -	Addition and subtraction
>>, <<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise exclusive 'OR' and regular 'OR'
<=, <, >, >=	Comparison operators
<>, ==, !=	Equality operators
=, %=, /=, //=, -=, +=, *=, **=	Assignment operators
is, is not	Identity operators
in, not in	Membership operators
not, or, and	Logical operators

Expressions in Python

- It is a combination of symbols (like variables, constants, and operators)
- Every programming language has its own set of rules that determines whether an expression is valid or not

Expressions in Python

- Valid:
 - `a=10`
 - `b=5`
 - `c=a*b`
 - `d=a+-b`
- Invalid:
 - `c=a+,-b`

Operations on Strings

- Like numbers, we can also manipulate strings by performing operations on them
 - Concatenation
 - String repetition
 - Slice a string

String Concatenation

- Like numbers, we can also add two strings in python
- Example,

```
Str1 = "The missile man of India"
```

```
Str2 = "- Dr. APJ Abdul Kalam"
```

```
Str3 = Str1 + Str2
```

```
Print(Str3)
```

The missile man of India- Dr. APJ Abdul Kalam

String Repetition

- Certain string can be multiplied by a number for repetition
- Str = "Hello "
- Str1 = Str*3
- print(Str1)
- Hello Hello Hello

Slice a String

- You can extract subsets of strings by using the slice operator ([] and [:])
- You need to specify index or the range of index of characters to be extracted
- The index of the first character is 0 and the last is n-1 where n is the number of characters in the string

Slice a String

```
#string slice
```

```
str = "Python is Easy !!!"
```

```
print(str)
```

```
print(str[0])
```

```
print([-1])
```

```
print(str[15:17])
```

```
print(str * 2)
```

```
print(str + "Isn't it?")
```

Type Conversion

- **In Python, it is just not possible to complete certain operations that involves different types of data.**
- **For example, it is not possible to perform "2" + 4 since one operand is an integer and the other is of string type.**

Type Conversion

```
>>>"20"+"30"  
'2030'
```

```
>>> int("2") + int("3")  
5
```

Function	Description
<code>int(x)</code>	Converts <code>x</code> to an integer
<code>long(x)</code>	Converts <code>x</code> to a long integer
<code>float(x)</code>	Converts <code>x</code> to a floating point number
<code>str(x)</code>	Converts <code>x</code> to a string
<code>tuple(x)</code>	Converts <code>x</code> to a tuple
<code>list(x)</code>	Converts <code>x</code> to a list
<code>set(x)</code>	Converts <code>x</code> to a set
<code>ord(x)</code>	Converts a single character to its integer value
<code>oct(x)</code>	Converts an integer to an octal string
<code>hex(x)</code>	Converts an integer to a hexadecimal string
<code>chr(x)</code>	Converts an integer to a character
<code>unichr(x)</code>	Converts an integer to a Unicode character
<code>dict(x)</code>	Creates a dictionary if <code>x</code> forms a (key-value) pair

Thank You

Dr. Abdul Kayom



Asst. Prof, (SENSE)



kayomabdul@vitap.ac.in



8474860025